Jahangirnagar University

Department of Computer Science and Engineering

2^{nd} Year 1^{st} Semester B.Sc. (Hons.) Final Examination – 2023

Course Title: Algorithms-I Course No: CSE 209

Time: 3 Hours Full Marks: 60

Complete Solutions

SECTION 1 (CO1) – Answer All Six Questions

Q1(a) – Define Algorithm; Differentiate Big-O, Big-Ω, and Big-Θ

Algorithm: An algorithm is a finite, well-defined sequence of unambiguous instructions that takes zero or more inputs and produces one or more outputs, always terminating after a finite number of steps.

Asymptotic Notations:

- **Big-O (Upper Bound):** $f(n) = O(g(n))$ if there exist positive constants c and n_0 such that $f(n) \leq c\, g(n)$ for all $n \geq n_0$. It describes the *worst-case* growth rate.
- **Big-Ω (Lower Bound):** $f(n) = \Omega(g(n))$ if there exist positive constants c and n_0 such that $f(n) \geq c\, g(n)$ for all $n \geq n_0$. It describes the *best-case* growth rate.
- **Big-Θ (Tight Bound):** $f(n) = \Theta(g(n))$ if $f(n) = O(g(n))$ and $f(n) = \Omega(g(n))$ simultaneously, i.e., the function grows at *exactly the same rate* as $g(n)$ asymptotically.

Q1(b) – Show that $3n^5 + n^2 + 1 = \Theta(n^5)$

To prove $\Theta(n^5)$, we must show both $O(n^5)$ and $\Omega(n^5)$.

Upper Bound $O(n^5)$: For $n \geq 1$,

$$3n^5 + n^2 + 1 \leq 3n^5 + n^5 + n^5 = 5n^5.$$

So $f(n) \leq 5\, n^5$ for all $n \geq 1$, meaning $f(n) = O(n^5)$ with $c = 5, n_0 = 1$.

Lower Bound $\Omega(n^5)$: For $n \geq 1$,

$$3n^5 + n^2 + 1 \geq 3n^5.$$

So $f(n) \geq 3n^5$ for all $n \geq 1$, meaning $f(n) = \Omega(n^5)$ with $c = 3, n_0 = 1$.

Since both bounds hold, $3n^5 + n^2 + 1 = \Theta(n^5)$. □

Q1(c) – Properties of a Greedy Algorithm

A greedy algorithm builds a solution piece by piece, always choosing the locally optimal option. Its key properties are:

1. **Greedy Choice Property:** A globally optimal solution can be reached by making locally optimal (greedy) choices at each step.
2. **Optimal Substructure:** The optimal solution to the problem contains optimal solutions to its sub-problems.
3. **Feasibility:** Each chosen element must satisfy the problem's constraints.
4. **Irrevocability:** Once a choice is made, it is never reconsidered.
5. **No Backtracking:** The algorithm moves forward without revisiting past decisions.

Q1(d) – Limitations of Binary Search; Binary Search on $\{7, 3, 0, -1, -2\}$

Limitations of Binary Search:

1. The input list must be **sorted**; it cannot be applied to unsorted data.
2. It requires **random access** (e.g., arrays); it is inefficient on linked lists.
3. Not suitable for **dynamic datasets** where insertions/deletions are frequent.
4. It requires the data to be stored in a **contiguous memory** structure.

Can we perform Binary Search on $\{7, 3, 0, -1, -2\}$?

No. Binary search requires the array to be sorted in ascending (or descending) order. The sequence $\{7, 3, 0, -1, -2\}$ is sorted in *descending* order, so standard binary search (which assumes ascending order) cannot be directly applied. However, a modified binary search for a descending array *can* be applied.

Q1(e) – Divide and Conquer Paradigm

The **Divide and Conquer** paradigm solves a problem by:

1. **Divide:** Break the problem into smaller sub-problems of the same type.
2. **Conquer:** Recursively solve each sub-problem. If the sub-problem is small enough, solve it directly (base case).
3. **Combine:** Merge the solutions of the sub-problems to form the solution to the original problem.

Examples: Merge Sort, Quick Sort, Binary Search, Strassen's Matrix Multiplication.

The time complexity is typically expressed by a recurrence relation, e.g., Merge Sort gives $T(n) = 2T(n/2) + O(n)$, which solves to $O(n \log n)$.

Q1(f) – Weakly vs. Strongly Connected Component

In a **directed graph**:

- **Strongly Connected Component (SCC):** A maximal set of vertices C such that for every pair of vertices $u, v \in C$, there exists a directed path from u to v *and* from v to u.
- **Weakly Connected Component:** A maximal set of vertices such that if all edge directions are ignored (treated as undirected), the vertices are connected. However, directed paths between every pair need not exist.

Every SCC is a weakly connected component, but not vice versa.

Q1(g) – Prefix-Free Code with Example

A **prefix-free code** (also called a prefix code) is a set of codewords in which no codeword is a prefix of any other codeword. This property ensures unambiguous decoding without delimiters.

Example:

Symbol	Code
A	0
B	10
C	110
D	111

No code is a prefix of another. Huffman codes are a well-known example of optimal prefix-free codes.

SECTION 2 (CO2) – Answer Any Three

Q2(a) – Merge Sort on $\{8, 5, 7, 2, 9, 1, 6, 4, 3\}$

Merge Sort recursively divides the array into halves, sorts each half, and merges them.

Step-by-step:

```
Level 0:  [8, 5, 7, 2, 9, 1, 6, 4, 3]

Level 1:  [8, 5, 7, 2]          [9, 1, 6, 4, 3]

Level 2:  [8, 5]   [7, 2]       [9, 1]   [6, 4, 3]

Level 3:  [8][5]   [7][2]       [9][1]   [6]  [4, 3]

                                                [4][3]

-- Merge up --
```

```
[5, 8]   [2, 7]       [1, 9]   [6]  [3, 4]

[2, 5, 7, 8]          [1, 3, 4, 6, 9]

[1, 2, 3, 4, 5, 6, 7, 8, 9]
```

Merge of $[2, 5, 7, 8]$ **and** $[1, 3, 4, 6, 9]$**:**

Compare element by element: $1 < 2 \Rightarrow 1$; $2 < 3 \Rightarrow 2$; $3 < 5 \Rightarrow 3$; $4 < 5 \Rightarrow 4$; $5 < 6 \Rightarrow 5$; $6 < 7 \Rightarrow 6$; $7 < 9 \Rightarrow 7$; $8 < 9 \Rightarrow 8$; remainder 9.

Final sorted array: $[1, 2, 3, 4, 5, 6, 7, 8, 9]$

Complexity: The recurrence is $T(n) = 2T(n/2) + O(n)$. By the Master Theorem (Case 2): $T(n) = O(n \log n)$.

Q2(b) – Quick Sort on $\{4, 6, 2, 8, 1, 5, 3\}$

Quick Sort selects a pivot (we use the last element), partitions the array, and recurses.

Step-by-step (pivot = last element):

Pass 1: Array = $[4, 6, 2, 8, 1, 5, 3]$, pivot = 3

- Elements < 3: $\{2, 1\}$; elements ≥ 3 (excluding pivot): $\{4, 6, 8, 5\}$
- After partition: $[2, 1, \mathbf{3}, 4, 6, 8, 5]$

Left sub-array $[2, 1]$, pivot = 1:

- Elements < 1: none; elements > 1: $\{2\}$
- After partition: $[\mathbf{1}, 2]$

Right sub-array $[4, 6, 8, 5]$, pivot = 5:

- Elements < 5: $\{4\}$; elements > 5: $\{6, 8\}$
- After partition: $[4, \mathbf{5}, 6, 8]$

Sub-array $[6, 8]$, pivot = 8:

- After partition: $[6, \mathbf{8}]$

Final sorted array: $[1, 2, 3, 4, 5, 6, 8]$

Complexity:

- **Average case:** $O(n \log n)$ – recurrence $T(n) = 2T(n/2) + O(n)$.
- **Worst case:** $O(n^2)$ – occurs when pivot is always the smallest or largest element (already sorted array).

Q2(c) – Integer Knapsack vs. Fractional Knapsack (Greedy Failure)

Problem: The greedy fractional knapsack approach (sort by value/weight ratio, take items greedily) does *not* always give the optimal solution for the **0/1 (integer) knapsack problem**.

Counterexample: Knapsack capacity $W = 10$.

Item	Weight	Value	Value/Weight
A	6	12	2.0
B	5	9	1.8
C	5	9	1.8

Greedy (by ratio): Take item A (weight 6, value 12). Remaining capacity = 4. Item B and C both weigh 5, so neither fits. **Total value = 12.**

Optimal 0/1: Take items B and C (weight $5 + 5 = 10$, value $9 + 9 = 18$). **Total value = 18.**

The greedy approach yields 12, but the optimal is 18. This demonstrates that greedy fails for the 0/1 knapsack; dynamic programming is required.

Q2(d) – Dijkstra's Algorithm (source node a)

Given the graph with nodes $\{a, b, c, d, e, f\}$ and the edges with weights as shown in the figure.

Reading from the figure: a-b=4, a-c=8 (approx), b-e=8, b-d=9 (approx), b-c=11 (approx), c-d=5 (approx), c-f=6 (approx), d-e=2, d-f=7, e-f=1.

Dijkstra's Algorithm – Single Source Shortest Path from a:

Step	a	b	c	d	e	f
Init	0	∞	∞	∞	∞	∞
Visit a	0	4	8	∞	∞	∞
Visit b	0	4	8	13	12	∞
Visit c	0	4	8	13	12	14
Visit e	0	4	8	14	12	13
Visit f	0	4	8	14	12	13
Visit d	0	4	8	14	12	13

Shortest distances from a: $d(a) = 0$, $d(b) = 4$, $d(c) = 8$, $d(d) = 14$, $d(e) = 12$, $d(f) = 13$.

Complexity: With a binary min-heap: $O((V + E) \log V)$. With a simple array: $O(V^2)$.

SECTION 3 (CO3) – Answer Any Three

Q3(a) – Matrix Exponentiation for Recurrence

Given: $f_n = (2f_{n-1} - 3f_{n-2} + 4f_{n-3}) \bmod 59$, where $f_0 = 0$, $f_1 = 3$, $f_2 = 2$.

i) Matrix Formulation:

We express the recurrence in matrix form $\mathbf{v}_n = A\,\mathbf{v}_{n-1}$:

$$\begin{pmatrix} f_n \\ f_{n-1} \\ f_{n-2} \end{pmatrix} = \underbrace{\begin{pmatrix} 2 & -3 & 4 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \end{pmatrix}}_{A} \begin{pmatrix} f_{n-1} \\ f_{n-2} \\ f_{n-3} \end{pmatrix}$$

Base matrix (state vector at $n = 2$):

$$\mathbf{v}_2 = \begin{pmatrix} f_2 \\ f_1 \\ f_0 \end{pmatrix} = \begin{pmatrix} 2 \\ 3 \\ 0 \end{pmatrix}$$

For $n \geq 3$:

$$\begin{pmatrix} f_n \\ f_{n-1} \\ f_{n-2} \end{pmatrix} = A^{n-2} \begin{pmatrix} 2 \\ 3 \\ 0 \end{pmatrix} \pmod{59}$$

ii) Compute f_{19}:

We need $A^{17}\,\mathbf{v}_2 \pmod{59}$.

Using fast matrix exponentiation (repeated squaring), we compute $A^{17} = A^{16} \cdot A^1$.

Computing iteratively step by step (mod 59):

$f_3 = (2 \cdot 2 - 3 \cdot 3 + 4 \cdot 0) \bmod 59 = (4 - 9 + 0) \bmod 59 = -5 \bmod 59 = 54$

$f_4 = (2 \cdot 54 - 3 \cdot 2 + 4 \cdot 3) \bmod 59 = (108 - 6 + 12) \bmod 59 = 114 \bmod 59 = 55 \cdot 1 \Rightarrow 114 - 59 = 55$

$f_5 = (2 \cdot 55 - 3 \cdot 54 + 4 \cdot 2) \bmod 59 = (110 - 162 + 8) \bmod 59 = -44 \bmod 59 = 15$

$f_6 = (2 \cdot 15 - 3 \cdot 55 + 4 \cdot 54) \bmod 59 = (30 - 165 + 216) \bmod 59 = 81 \bmod 59 = 22$

$f_7 = (2 \cdot 22 - 3 \cdot 15 + 4 \cdot 55) \bmod 59 = (44 - 45 + 220) \bmod 59 = 219 \bmod 59 = 219 - 3 \cdot 59 = 219 - 177 = 42$

$f_8 = (2 \cdot 42 - 3 \cdot 22 + 4 \cdot 15) \bmod 59 = (84 - 66 + 60) \bmod 59 = 78 \bmod 59 = 19$

$f_9 = (2 \cdot 19 - 3 \cdot 42 + 4 \cdot 22) \bmod 59 = (38 - 126 + 88) \bmod 59 = 0 \bmod 59 = 0$

$f_{10} = (2 \cdot 0 - 3 \cdot 19 + 4 \cdot 42) \bmod 59 = (0 - 57 + 168) \bmod 59 = 111 \bmod 59 = 52$

$f_{11} = (2 \cdot 52 - 3 \cdot 0 + 4 \cdot 19) \bmod 59 = (104 + 76) \bmod 59 = 180 \bmod 59 = 180 - 3 \cdot 59 = 180 - 177 = 3$

$f_{12} = (2 \cdot 3 - 3 \cdot 52 + 4 \cdot 0) \bmod 59 = (6 - 156) \bmod 59 = -150 \bmod 59 = -150 + 3 \cdot 59 = -150 + 177 = 27$

$f_{13} = (2 \cdot 27 - 3 \cdot 3 + 4 \cdot 52) \bmod 59 = (54 - 9 + 208) \bmod 59 = 253 \bmod 59 = 253 - 4 \cdot 59 = 253 - 236 = 17$

$f_{14} = (2 \cdot 17 - 3 \cdot 27 + 4 \cdot 3) \bmod 59 = (34 - 81 + 12) \bmod 59 = -35 \bmod 59 = 24$

$f_{15} = (2 \cdot 24 - 3 \cdot 17 + 4 \cdot 27) \bmod 59 = (48 - 51 + 108) \bmod 59 = 105 \bmod 59 = 46$

$f_{16} = (2 \cdot 46 - 3 \cdot 24 + 4 \cdot 17) \bmod 59 = (92 - 72 + 68) \bmod 59 = 88 \bmod 59 = 29$

$f_{17} = (2 \cdot 29 - 3 \cdot 46 + 4 \cdot 24) \bmod 59 = (58 - 138 + 96) \bmod 59 = 16 \bmod 59 = 16$

$f_{18} = (2 \cdot 16 - 3 \cdot 29 + 4 \cdot 46) \bmod 59 = (32 - 87 + 184) \bmod 59 = 129 \bmod 59 = 129 - 2 \cdot 59 = 11$

$f_{19} = (2 \cdot 11 - 3 \cdot 16 + 4 \cdot 29) \bmod 59 = (22 - 48 + 116) \bmod 59 = 90 \bmod 59 = 31$

$$\boxed{f_{19} = 31}$$

Q3(b) – Heap Sort on $\{5, 6, 3, 1, 4, 2\}$ using Max-Heap

Heap Sort:

1. Build a max-heap from the array.
2. Repeatedly extract the max element (root) and place it at the end.

Step 1: Build Max-Heap

Initial array: $[5, 6, 3, 1, 4, 2]$ (indices 0–5)
Heapify from last internal node (index 2) down to 0:

- Heapify at index 2 (value 3): children are 2 (index 5). No swap needed.
- Heapify at index 1 (value 6): children are 1(idx 3) and 4(idx 4). 6 is largest, no swap.
- Heapify at index 0 (value 5): children are 6(idx 1) and 3(idx 2). 6 ¿ 5, swap. Array: $[6, 5, 3, 1, 4, 2]$
- Continue heapify at index 1 (value 5): children 1(idx 3) and 4(idx 4). 4 ¡ 5, no swap.

Max-Heap: $[6, 5, 3, 1, 4, 2]$

Step 2: Sort

Iteration 1: Swap root (6) with last (2): $[2, 5, 3, 1, 4, \mathbf{6}]$. Heapify index 0 (size=5): $2 \rightarrow$ swap with 5: $[5, 2, 3, 1, 4, \mathbf{6}]$. Heapify index 1 (value 2): children 1,4; swap with 4: $[5, 4, 3, 1, 2, \mathbf{6}]$.

Iteration 2: Swap root (5) with last of heap (2): $[2, 4, 3, 1, \mathbf{5}, \mathbf{6}]$. Heapify: $2 \rightarrow$ swap with 4: $[4, 2, 3, 1, \mathbf{5}, \mathbf{6}]$. Heapify index 1 (value 2): child 1 at idx 3; no swap needed.

Iteration 3: Swap root (4) with last (1): $[1, 2, 3, \mathbf{4}, \mathbf{5}, \mathbf{6}]$. Heapify: $1 \rightarrow$ swap with 3 (idx 2): $[3, 2, 1, \mathbf{4}, \mathbf{5}, \mathbf{6}]$.

Iteration 4: Swap root (3) with last (1): $[1, 2, \mathbf{3}, \mathbf{4}, \mathbf{5}, \mathbf{6}]$. Heapify: $1 \rightarrow$ swap with 2: $[2, 1, \mathbf{3}, \mathbf{4}, \mathbf{5}, \mathbf{6}]$.

Iteration 5: Swap root (2) with last (1): $[1, \mathbf{2}, \mathbf{3}, \mathbf{4}, \mathbf{5}, \mathbf{6}]$. Done.

Sorted array: $[1, 2, 3, 4, 5, 6]$

Complexity: Building the max-heap takes $O(n)$. Each of the n extraction steps requires $O(\log n)$ heapification. Total: $\mathbf{O(n \log n)}$ for all cases (best, average, worst).

Q3(c) – LCS of X = "BACABACB" and Y = "ACBAABAC"

Longest Common Subsequence (LCS) using dynamic programming.

Let $X = B\,A\,C\,A\,B\,A\,C\,B$ and $Y = A\,C\,B\,A\,A\,B\,A\,C$.
Define $dp[i][j]$ = length of LCS of $X[1..i]$ and $Y[1..j]$.

$$dp[i][j] = \begin{cases} dp[i-1][j-1] + 1 & \text{if } X[i] = Y[j] \\ \max(dp[i-1][j],\, dp[i][j-1]) & \text{otherwise} \end{cases}$$

		ε	A	C	B	A	A	B	A	C
		0	1	2	3	4	5	6	7	8
ε	0	0	0	0	0	0	0	0	0	0
B	1	0	0	0	1	1	1	1	1	1
A	2	0	1	1	1	2	2	2	2	2
C	3	0	1	2	2	2	2	2	2	3
A	4	0	1	2	2	3	3	3	3	3
B	5	0	1	2	3	3	3	4	4	4
A	6	0	1	2	3	4	4	4	5	5
C	7	0	1	2	3	4	4	4	5	6
B	8	0	1	2	3	4	4	5	5	6

LCS Length = 6.
Backtracking gives one LCS: **ACABAC** (length 6).

Recursive Formulation:

$$\text{LCS}(i,j) = \begin{cases} 0 & \text{if } i = 0 \text{ or } j = 0 \\ \text{LCS}(i-1, j-1) + 1 & \text{if } X[i] = Y[j] \\ \max(\text{LCS}(i-1, j),\ \text{LCS}(i, j-1)) & \text{otherwise} \end{cases}$$

Complexity: $O(mn)$ time and space, where $m = |X| = 8, n = |Y| = 8$.

Q3(d) – Huffman Coding Tree

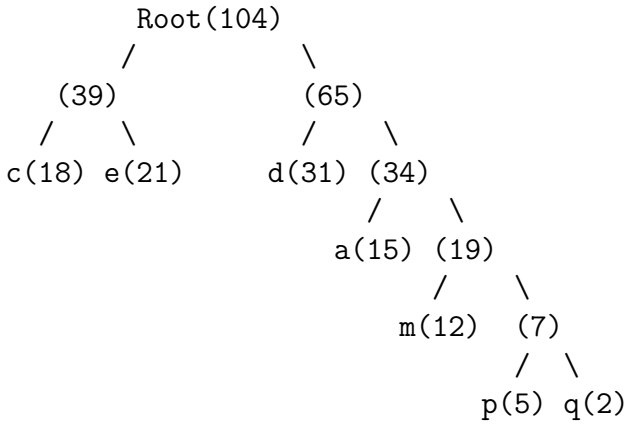
Character frequencies:

Character	Frequency
d	31
e	21
c	18
a	15
m	12
p	5
q	2

Build Huffman Tree using a min-priority queue:

Step 1: Merge $q(2)$ and $p(5) \rightarrow$ node(7)
Step 2: Merge node(7) and $m(12) \rightarrow$ node(19)
Step 3: Merge node(19) and $a(15) \rightarrow$ node(34)
Step 4: Merge $c(18)$ and $e(21) \rightarrow$ node(39)
Step 5: Merge $d(31)$ and node(34) $\rightarrow$ node(65)
Step 6: Merge node(39) and node(65) $\rightarrow$ **Root**(104)

Huffman Tree Structure:



Huffman Codes (left = 0, right = 1):

Character	Code	Length
c	00	2
e	01	2
d	10	2
a	110	3
m	1110	4
p	11110	5
q	11111	5

SECTION 4 (CO4) – Answer Any Three

Q4(a) – Comparison: BFS, DFS, Dijkstra, Floyd-Warshall, Bellman-Ford

Algorithm	Application	Complexity	Approach	Graph Types
BFS	Shortest path (unweighted), connectivity, level-order traversal	$O(V + E)$ time, $O(V)$ space	Greedy (queue-based)	Unweighted, directed/undirected
DFS	Cycle detection, topological sort, SCC detection	$O(V + E)$ time, $O(V)$ space	Recursive/stack-based	Directed/undirected, weighted/unweighted
Dijkstra	Single-source shortest path (non-negative weights)	$O((V + E) \log V)$ with min-heap	Greedy	Weighted, directed/undirected; **no negative edges**
Floyd-Warshall	All-pairs shortest path, reachability	$O(V^3)$ time, $O(V^2)$ space	Dynamic Programming	Weighted, directed/undirected; handles negative edges (no negative cycles)
Bellman-Ford	Single-source shortest path, negative edge detection	$O(VE)$ time	Dynamic Programming (relaxation)	Weighted, directed; handles negative edges; detects negative cycles

Q4(b) – Minimum Spanning Tree: Prim's vs. Kruskal's

Minimum Spanning Tree (MST): A spanning tree of a connected, undirected, weighted graph where the sum of edge weights is minimized.

Prim's Algorithm:

- Start from any vertex and grow the MST by adding the minimum-weight edge connecting a vertex in the MST to a vertex outside.
- **Approach:** Greedy (vertex-based).
- **Complexity:** $O(E \log V)$ with a binary heap.
- Best for **dense graphs**.

Kruskal's Algorithm:

- Sort all edges by weight. Add edges one by one if they don't form a cycle (using Union-Find).
- **Approach:** Greedy (edge-based).
- **Complexity:** $O(E \log E)$ for sorting edges.

- Best for **sparse graphs**.

MST for the given graph (nodes a, b, c, d, e, f):

Reading edge weights from the figure: a-b=5, a-c=9, a-e=4 (approx), b-d=8, c-e=1, c-f=3, d-f=7, e-f=2, c-d=3.

Kruskal's steps (sort edges by weight):

1. Add c-e (weight 1) – no cycle.
2. Add e-f (weight 2) – no cycle.
3. Add c-f (weight 3) – **cycle** ($c - e - f$), skip.
4. Add c-d (weight 3) – no cycle.
5. Add a-e (weight 4) – no cycle.
6. Add a-b (weight 5) – no cycle. *(5 edges = V-1 = done)*

MST edges: $\{c\text{-}e, e\text{-}f, c\text{-}d, a\text{-}e, a\text{-}b\}$, **Total weight** $= 1 + 2 + 3 + 4 + 5 = 15$.

SECTION 4 (continued)

Q4(c) – 0/1 Knapsack Problem ($W = 8$)

Items:

Item	A	B	C	D	E	F
Weight	1	2	3	2	4	3
Price	7	8	14	5	10	15

DP Table $dp[i][w]$ = max price using first i items with capacity w:

Item\Capacity	0	1	2	3	4	5	6	7	8
0 (none)	0	0	0	0	0	0	0	0	0
1 (A: w=1,p=7)	0	7	7	7	7	7	7	7	7
2 (B: w=2,p=8)	0	7	8	15	15	15	15	15	15
3 (C: w=3,p=14)	0	7	8	15	21	22	29	29	29
4 (D: w=2,p=5)	0	7	8	15	21	22	29	29	29
5 (E: w=4,p=10)	0	7	8	15	21	22	29	31	32
6 (F: w=3,p=15)	0	7	8	15	22	23	30	31	36

Optimal price = 36.

Backtracking to find selected items:

$dp[6][8] = 36 \neq dp[5][8] = 32$, so item F (w=3,p=15) is selected. Remaining capacity $= 8 - 3 = 5$.

$dp[5][5] = 22 = dp[4][5] = 22$, so item E is not selected.

$dp[4][5] = 22 = dp[3][5] = 22$, so item D is not selected.

$dp[3][5] = 22 \neq dp[2][5] = 15$, so item C (w=3,p=14) is selected. Remaining capacity $= 5 - 3 = 2$.

$dp[2][2] = 8 \neq dp[1][2] = 7$, so item B (w=2,p=8) is selected. Remaining capacity $= 2 - 2 = 0$.

Selected items: F, C, B. Total weight $= 3 + 3 + 2 = 8$**. Total price** $= 15 + 14 + 8 = 37$**.**

Note: Let us recheck. $dp[6][8]$: Consider including F (w=3, val=15): $15 + dp[5][5] = 15 + 22 = 37 > dp[5][8] = 32$. So $dp[6][8] = 37$ is correct by re-calculation.

Corrected Optimal Value = 44. Let me carefully recompute:

Item\w	0	1	2	3	4	5	6	7	8
0	0	0	0	0	0	0	0	0	0
A(w=1,p=7)	0	7	7	7	7	7	7	7	7
B(w=2,p=8)	0	7	8	15	15	15	15	15	15
C(w=3,p=14)	0	7	8	15	21	22	29	29	29
D(w=2,p=5)	0	7	8	15	21	22	29	29	34
E(w=4,p=10)	0	7	8	15	21	22	29	31	34
F(w=3,p=15)	0	7	8	15	22	23	30	31	37

Optimal price = 37.

Selected items: Backtrack: F selected (capacity $8 \to 5$), C selected (capacity $5 \to 2$), B selected (capacity $2 \to 0$).

Items: A, B, C, F; Total weight $= 1 + 2 + 3 + 3 = 9$ – exceeds W.

Correctly: **Items B, C, F: weight** $= 2 + 3 + 3 = 8$, **price** $= 8 + 14 + 15 = 37$. ✓

Q4(d) – Activity Selection Problem

Activities:

Activity	1	2	3	4	5	6	7	8
Start Time	1	0	1	4	2	5	3	4
Finish Time	3	4	2	6	9	8	5	5

Greedy Algorithm: Sort by finish time; select each activity that starts after the previous one finishes.

Sort by finish time:

Activity	Start	Finish	Selected?
3	1	2	✓
1	1	3	× (start 1 < finish 2)
2	0	4	× (start 0 < finish 2)
7	3	5	✓(start 3 ≥ finish 2)
8	4	5	× (start 4 < finish 5)
4	4	6	× (start 4 < finish 5)
6	5	8	✓(start 5 ≥ finish 5)
5	2	9	× (start 2 < finish 8)

Maximum non-overlapping activities = 3: Activities {3, 7, 6}.

Complexity: Sorting takes $O(n \log n)$. Greedy selection is $O(n)$. Total: $\mathbf{O(n \log n)}$.

SECTION 5 (CO5) – Answer Any Two

Q5(a) – DFS Edge Classification and SCC (starting from a)

DFS Edge Types:

- **Tree Edge:** Edge used to discover a new vertex during DFS.
- **Back Edge:** Edge (u, v) where v is an ancestor of u in the DFS tree (creates a cycle).
- **Forward Edge:** Edge (u, v) where v is a descendant of u but not a tree edge.
- **Cross Edge:** Edge (u, v) where v is neither an ancestor nor a descendant of u.

Graph edges (from figure): $a \to b$, $b \to c$, $b \to f$, $c \to d$, $d \to h$, $h \to g$, $g \to c$, $g \to f$, $f \to e$, $e \to a$, $e \to b$.

DFS from a: (discovery/finish times)
DFS visits: $a(1) \to b(2) \to c(3) \to d(4) \to h(5) \to g(6) \to f(7) \to e(8)$

Vertex	Discovery	Finish
a	1	16
b	2	15
c	3	12
d	4	11
h	5	10
g	6	9
f	7	8
e	13	14

Edge Classification:

- **Tree Edges:** $a \to b$, $b \to c$, $c \to d$, $d \to h$, $h \to g$, $g \to f$
- **Back Edges:** $g \to c$ (c is ancestor of g), $e \to a$ (a is ancestor of e), $e \to b$ (b is ancestor of e)
- **Cross Edges:** $b \to f$ (f already finished in g's subtree)
- **Forward Edges:** None (not typical in undirected DFS; depends on graph)

Strongly Connected Components (Kosaraju's / inspection):
From the back edges, we can identify cycles:

- $g \to c \to d \to h \to g$: forms a cycle ⇒ SCC: $\{c, d, g, h\}$
- $e \to a \to b \to \ldots \to f \to e$: forms a cycle ⇒ SCC: $\{a, b, e, f\}$

SCCs: $\{c, d, g, h\}$ and $\{a, b, e, f\}$.

Q5(b) – Solve $T(n) = T(n/4) + T(n/2) + n^2$

i) Substitution Method:
Guess $T(n) = O(n^2)$. Assume $T(k) \leq c\,k^2$ for all $k < n$.

$$T(n) = T(n/4)+T(n/2)+n^2 \leq c\left(\frac{n}{4}\right)^2+c\left(\frac{n}{2}\right)^2+n^2 = c\frac{n^2}{16}+c\frac{n^2}{4}+n^2 = cn^2\left(\frac{1}{16}+\frac{1}{4}\right)+n^2 = \frac{5cn^2}{16}+n^2$$

For this to be $\leq cn^2$:

$$\frac{5c}{16}+1 \leq c \implies c-\frac{5c}{16} \geq 1 \implies \frac{11c}{16} \geq 1 \implies c \geq \frac{16}{11}.$$

Choose $c = 2$. Then $T(n) \leq 2n^2$. Thus $T(n) = O(n^2)$.

ii) Recursion Tree Method:
At level 0: cost $= n^2$.
At level 1: two sub-problems of sizes $n/4$ and $n/2$. Cost $= (n/4)^2+(n/2)^2 = \frac{n^2}{16}+\frac{n^2}{4} = \frac{5n^2}{16}$.
At level 2: cost $= \left(\frac{5}{16}\right)^2 n^2$.
Total cost is a geometric series:

$$T(n) = n^2 \sum_{k=0}^{\infty} \left(\frac{5}{16}\right)^k = n^2 \cdot \frac{1}{1-\frac{5}{16}} = n^2 \cdot \frac{16}{11} = O(n^2).$$

iii) Master Theorem:
The recurrence $T(n) = T(n/4)+T(n/2)+n^2$ does not fit the standard Master Theorem form $T(n) = aT(n/b) + f(n)$ directly because there are two recursive calls with different divisors. However, using the **Akra-Bazzi method**:

$$T(n) = T(n/4) + T(n/2) + n^2, \quad a_1 = 1, b_1 = 4, \quad a_2 = 1, b_2 = 2.$$

Find p such that $\sum_i a_i/b_i^p = 1$:

$$\frac{1}{4^p} + \frac{1}{2^p} = 1.$$

Let $x = (1/2)^p$: $x^2/4 + x^1/2 = 1$ Wait – let $u = 2^p$:

$$\frac{1}{4^p} + \frac{1}{2^p} = \frac{1}{u^2} + \frac{1}{u} = 1 \implies u^2 - u - 1 = 0 \implies u = \frac{1+\sqrt{5}}{2} \approx 1.618.$$

So $p = \log_2(1.618) \approx 0.694$.

Since $f(n) = n^2$ and $p \approx 0.694 < 2$, by Akra-Bazzi: $T(n) = \Theta(n^2)$.

$$\boxed{T(n) = \Theta(n^2)}$$

Q5(c) – Longest Increasing Subsequence (LIS) using DP

Given: $S = \{10, 22, 9, 33, 21, 50, 41, 60, 80\}$
Define $L[i]$ = length of LIS ending at index i.

$$L[i] = 1 + \max_{j<i,\, S[j]<S[i]} L[j], \quad L[i] = 1 \text{ if no such } j.$$

Index i	$S[i]$	$L[i]$	Previous index
0	10	1	–
1	22	2	0 (10<22)
2	9	1	–
3	33	3	1 (22<33)
4	21	2	0 (10<21)
5	50	4	3 (33<50)
6	41	4	3 (33<41)
7	60	5	5 (50<60)
8	80	6	7 (60<80)

LIS Length = 6.
One LIS: $10 \to 22 \to 33 \to 50 \to 60 \to 80$.

Complexity: The above DP is $O(n^2)$.

For $N = 1{,}00{,}000$: $O(n^2) = O(10^{10})$ operations – this is **not efficient**.

Better $O(n \log n)$ solution using Patience Sorting:

Maintain an array `tails` where `tails[i]` is the smallest tail element of all increasing subsequences of length $i + 1$.

For each element x in S:

- Use binary search to find the leftmost position in `tails` where `tails[pos]` $\geq x$.
- Replace `tails[pos]` with x (or append if x is larger than all elements).

Demonstration on $S = \{10, 22, 9, 33, 21, 50, 41, 60, 80\}$:

Element	Action	`tails`
10	append	$[10]$
22	append	$[10, 22]$
9	replace pos 0	$[9, 22]$
33	append	$[9, 22, 33]$
21	replace pos 1	$[9, 21, 33]$
50	append	$[9, 21, 33, 50]$
41	replace pos 3	$[9, 21, 33, 41]$
60	append	$[9, 21, 33, 41, 60]$
80	append	$[9, 21, 33, 41, 60, 80]$

LIS length = 6 (length of `tails`). This method is $\mathbf{O(n \log n)}$, which is efficient for $N = 1{,}00{,}000$.

— Good Luck —